

Cache-Aware Request Routing for Distributed RAG-Based LLM Serving: Design, Calibration, and Empirical Findings

Saadet Cansu Baktiroğlu
Dept. of Artificial Intelligence
Engineering
TOBB University
of Economics and Technology
Student ID: 231401023
sbaktiroglu@etu.edu.tr

Umut Baran Boztaş
Dept. of Computer Engineering
TOBB University
of Economics and Technology
Student ID: 231101064
uboztas@etu.edu.tr

Demir Doruk Dilek
Dept. of Computer Engineering
TOBB University
of Economics and Technology
Student ID: 231104080
demirdoruk.dilek@etu.edu.tr

Abstract—Retrieval-augmented generation (RAG) workloads served across a pool of large language model (LLM) replicas create a routing problem that plain load balancing ignores: which replica already holds the retrieved context in its KV prefix cache. We present a cache-aware request router placed transparently in front of an OpenAI-API-compatible vLLM deployment, and a family of seven interchangeable routing strategies unified under one scoring framework, ranging from cache-blind baselines to a worker-independent reordering tree with two pluggable overlap-adaptive weighting mechanisms (a simple threshold switch and an EWMA/CUSUM drift detector) and a semantic pre-filter that narrows the candidate replica set before an $O(k^2)$ reorder step. Every heuristic constant in the system (load reference points, prompt-token estimates, SLO thresholds, drift targets) is derived from on-hardware measurement rather than borrowed defaults. We report several results that would have been missed without this calibration discipline: recalibrating a single tracker-capacity constant reverses the direction of a core ablation’s conclusion; a silent tie-break bias in a dual-hash baseline produces a load-balance asymmetry that looks like a modelling artifact but is a one-line code defect; and recording cache state at request completion rather than at dispatch time – the theoretically “safer” choice – measurably underperforms once queueing effects are accounted for. All strategies are validated both in isolated, GPU-free unit tests and on a live two-worker deployment.

Index Terms—LLM serving, KV cache, prefix caching, request routing, retrieval-augmented generation, load balancing, vLLM

I. INTRODUCTION

Serving large language models behind a pool of replicas is now standard practice for throughput and availability. When the workload is retrieval-augmented – each request’s prompt is built from a fixed system preamble followed by a handful of retrieved document chunks – the serving engine’s prefix KV cache can eliminate most of the prefill cost for a request *if* it is routed to a replica that already holds a matching prefix in cache. A router that is blind to this fact (round robin, or load balancing on queue depth alone) discards a large, freely available source of latency reduction: which replica currently holds which prefix is entirely a function of routing history, so

the router itself controls the very signal it would need to route well.

This paper describes a cache-aware router built as a transparent reverse proxy in front of N vLLM [1] replicas, together with a progression of routing strategies of increasing sophistication. The starting point is a single scoring function,

$$\text{score}(w) = \alpha \cdot \text{cache_gain}(w) - \beta \cdot \text{load}(w),$$

of which round-robin, pure load balancing, and a cache-aware policy with an adaptive guard band are all degenerate special cases ($\alpha=\beta=0$ plus rotation; $\alpha=0$; and the full form, respectively). We extend this baseline in four largely independent directions and ablate each one:

- 1) **Worker-independent joint reordering.** A per-worker variant of the CacheWeaver [2] knowledge tree replaces DualMap’s [3] dual-hash candidate selection, which we show degenerates to a no-op at $n=2$ replicas – exactly the deployment size used throughout this work.
- 2) **Overlap-adaptive weighting.** Two independent mechanisms – a threshold switch and an EWMA+CUSUM drift detector – convert a live, consecutive-request Jaccard-overlap signal into a dynamic (α, β) pair, and are ablated against each other under a *controlled* design: both consume the same input signal, so any difference between them is attributable to the adaptation mechanism alone.
- 3) **Semantic candidate pre-filtering.** A lightweight, pre-retrieval embedding classifier narrows the candidate replica set from N to $k \ll N$ before the $O(k^2)$ reorder step is paid, addressing a gap we identify in the routing literature (Section II).
- 4) **Calibration.** Every constant that would otherwise be a guess – the load-normalisation reference point, the average prompt-token estimate feeding a token-budget-aware baseline, SLO and rebalancing thresholds, and the drift detector’s target overlap level – is measured on the

actual two-machine deployment rather than copied from a paper’s own hardware.

The calibration discipline turns out not to be a formality. Section VI reports a case where recalibrating a single constant (TRACKER_CAPACITY) reverses which of two policies wins a head-to-head comparison at every load level tested, and a case where a one-line tie-break defect in a baseline router silently locked a $\sim 67/33$ worker split for the same reason that other parts of this codebase deliberately randomise ties.

The question at the centre of this work can be stated directly: in a distributed LLM serving cluster, can reordering a RAG request’s retrieved document set for cache-friendliness be optimised *jointly* with the worker-selection decision, so that cache-affinity and load-balancing are pursued together rather than traded off – and, critically, *under which traffic conditions* (which retrieved-document-overlap regime) does that joint optimisation actually pay off, and under which does it disappear? The second half of that question matters as much as the first: this paper deliberately asks *under what conditions* the approach wins rather than claiming it wins unconditionally, in keeping with CacheWeaver’s [2] own finding that reordering gains depend strongly on document-overlap rate and vanish in low-overlap regimes such as HotpotQA.

II. RELATED WORK

Three largely separate literature threads intersect with this work: distributed cache-affinity routing, RAG-specific cache reuse, and adaptive/semantic parameter tuning. Of the 18 papers reviewed in this project’s literature survey, 15 are cited directly below (four implemented against or matched feature-for-feature, the remainder positioned in Table I); vLLM [1] underlies the serving engine itself and is discussed separately at the end of this section.

A. Distributed Cache-Affinity Routing

Preble [5] routes requests sharing a prefix to the same replica to maximise KV-cache reuse, maintaining a precise prefix tree per replica and switching from prompt-aware to load-aware scheduling once a request’s prefix-hit rate falls below a threshold. Mooncake [6] disaggregates prefill and decode into separate clusters around a KVCache-centric scheduler that balances throughput against latency SLOs. NVIDIA Dynamo [7] implements a production KV-cache-aware router that scores replicas on a combined decode-cost/prefill-cost estimate derived from cache overlap. DualMap [3] identifies a shared weakness across this line of work – oscillating between a single cache-affinity objective and a single load-balancing objective in one shared mapping space – and proposes a “power of two choices”-style fix: each request is mapped to two independent candidate replicas via two independent hashes, with SLO-aware routing and hotspot-aware rebalancing arbitrating between them.

This thread solves worker selection but is *not RAG-specific*: it treats the prefix as a fixed session history and does not address the fact that retrieval returns a *set* of documents whose

order can differ from one query to the next even when the set itself overlaps heavily with a previous query’s.

B. RAG-Specific Cache Reuse

RAGCache [8] builds a knowledge tree over retrieved document sequences and caches intermediate states across the GPU/host memory hierarchy, additionally overlapping retrieval with speculative generation. CacheBlend [9] and TurboRAG [10] target the case where reusable chunks do *not* form a contiguous prefix at all, fusing precomputed KV caches (selectively recomputing only a small subset of tokens to preserve generation quality) rather than relying on prefix alignment. Cache-Craft [11] shows that plain prefix caching degrades substantially on production RAG traffic and proposes identifying which chunk-caches remain valid, cheaply repairing the ones that do not. CacheWeaver [2] – the mechanism this paper’s `per_worker_tree` strategy builds on – takes a different, engine-unmodified approach: a lightweight prompt-layer method that reorders retrieved chunks into a cache-friendly sequence via a greedy prefix-tree walk (its Algorithm 1), reported to reach 97.5% of an oracle ordering’s reusable-prefix depth.

This thread solves RAG-specific reordering but *assumes a single serving instance*: none of these systems include a distributed worker-selection component.

C. Adaptive Tuning and Semantic Approaches

GORGO [12] models per-request cost as the sum of network, queueing, and prefill terms and tunes routing weights online via an evolutionary strategy against a held-out tuning window; it is network-topology-aware but not RAG-specific and requires black-box search rather than a closed-form update. The semantic-caching family – GPTCache [13], SCALM [14], and MeanCache [15] – compares query embeddings and, on a sufficiently similar match, returns a cached *answer* directly, skipping computation entirely; this operates one layer above KV-cache reuse or worker routing and carries a distinct risk profile (a false-positive match returns a wrong answer outright, rather than merely costing extra prefill).

D. Positioning of This Work

TABLE I
RELATED WORK COMPARED ALONG THREE AXES

System	Dist. routing	RAG reorder	Adaptive
Preble [5]	✓	×	partial (threshold)
Mooncake [6]	✓	×	×
Dynamo [7]	✓	×	×
DualMap [3]	✓	×	×
RAGCache [8]	×	✓	×
CacheBlend [9]	×	✓	×
TurboRAG [10]	×	✓	×
Cache-Craft [11]	×	✓	×
CacheWeaver [2]	×	✓	×
GORGO [12]	✓	×	✓ (evolutionary)
Semantic caching	×	×	×
This work	✓	✓	✓ (overlap-adaptive)

This paper combines distributed routing (Section III-A/B/D) with RAG-specific reordering (Section III-C) *in one system*: retrieval output is first reordered for cache friendliness, and that ordering is then evaluated per-replica and optimised jointly with worker selection, rather than as two independent stages. To our knowledge no system in the surveyed literature addresses this specific dependency – whether to reorder before or after choosing a worker changes which ordering is actually optimal, since two replicas’ caches generally disagree on what is cache-friendly (Section III-C). Techniques from Section III-C above (adaptive tuning, semantic selection) are borrowed as extended evaluation components layered on top of this core design, not presented as this paper’s primary contribution.

RAGRoute [4] and Adaptive-RAG are related in spirit but orthogonal in target: both demonstrate that a cheap classifier run *before* retrieval – deciding which data source or retrieval strategy to use in under 30 ms – measurably improves federated RAG pipelines, but neither applies this idea to *which replica’s KV cache* is relevant to an incoming query, decided before the retrieval step that would otherwise be needed to know which chunks – and hence which cache – are relevant at all. Section III-F describes the semantic pre-filter built to close this specific gap.

Serving infrastructure. vLLM [1] provides the underlying PagedAttention KV-cache engine and the OpenAI-compatible API surface this router proxies. All prefix-cache accounting in this work is calibrated against vLLM’s own reported counters (prompt_tokens_details.cached_tokens and the Prometheus vllm:prefix_cache_* series) rather than assumed.

III. SYSTEM DESIGN

A. Architecture

The router is a FastAPI reverse proxy exposing the same /v1/chat/completions, /v1/completions, and /v1/models surface as vLLM itself, so that a client (in our deployment, an Open WebUI instance) cannot distinguish it from talking to a bare engine – streaming responses are forwarded byte-for-byte rather than buffered. A background poller scrapes each worker’s /metrics endpoint on a fixed interval and maintains a per-worker WorkerState snapshot (queue depth, running-request count, KV-cache utilisation, prefix-cache hit counters); routing decisions read this snapshot rather than querying workers synchronously on the request path.

Because vLLM exposes aggregate cache statistics but not "do you currently hold a cache entry for this specific prefix", the router keeps its own approximation: for each worker, an LRU of block-aligned prefix hashes it has been sent (PrefixTracker), hashed in the same fixed token-block granularity vLLM itself uses. Two independent bookkeeping conventions are supported for *when* a block is recorded as cached – at dispatch time (optimistic: the block will be resident once prefill starts) or at completion time (conservative: only once the engine has actually finished) – and Section VI-E reports the result of ablating between them.

B. Strategy Framework

All strategies implement a single interface, `select(context, snapshot) -> Decision`, so that switching the active policy is a configuration change rather than a code change. The five directly comparable baselines are:

- **round_robin**: $\alpha=\beta=0$, plus a rotating tie-break; the floor every other strategy is measured against.
- **least_loaded**: $\alpha=0$; pure, cache-blind load balancing on a saturating load-normalisation $\text{load_norm}(w) = \text{load}(w)/(\text{load}(w) + \text{LOAD_REF})$, chosen over a clipped $\min(\cdot, 1)$ form because clipping destroys the policy’s discriminative power once both replicas exceed the reference load (Section IV-B).
- **cache_aware**: the full scoring function plus an *adaptive guard band*, $\delta = \delta_0 \cdot (1 - \text{kv_usage})$; the winning replica by score is overridden back to the least-loaded replica if its load exceeds the minimum by more than δ . This prevents the classic failure mode of pure cache-affinity routing, where the replica holding a popular prefix keeps attracting traffic until its queue makes it the worst choice by latency despite remaining the best choice by cache.
- **adaptive_cache_aware**: cache_aware with β and δ_0 driven live by an EWMA/CUSUM drift detector (Section III-E) observing session-adjacent retrieved-chunk-set overlap, rather than the fixed constants above.
- **cacheweaver_dualmap**: the DualMap-style baseline described in Section III-D.

C. Worker-Independent (per_worker_tree)

Joint

Reordering

DualMap’s replica candidate-selection step (its Eq. 5) hashes the reordered chunk sequence through two independent consistent-hashing rings to obtain a candidate pair (r_1, r_2) , with a collision rule $r_2 \leftarrow (r_1 + 1) \bmod n$ when the two rings agree. At $n=2$ replicas – the deployment size used throughout this work – this collision rule is not an edge case: it is provably always triggered or already trivially satisfied, so $\{r_1, r_2\} = \{0, 1\}$ on every request. The dual-hash candidate-selection machinery is therefore a mathematically guaranteed no-op at this scale, and any cache-affinity signal has to come from elsewhere in the pipeline.

`per_worker_tree` replaces the hash-and-select step entirely. Each worker maintains its own independent CacheWeaver knowledge tree; for every candidate worker, `best_reorder_for()` computes both that worker’s own best reordering of the retrieved chunks *and* a normalised cache-gain figure against that specific tree, and the scoring function from Section III-B picks the winner directly – there is no candidate-pruning step to degenerate. This is a genuine change in what "the right chunk order" means: it is no longer a single, worker-independent quantity computed once and then assigned to whichever replica is chosen, but something that emerges jointly with the worker decision, since the two replicas’ trees generally disagree about which order is cache-friendly.

Because the winner and its optimal chunk order are decided together, a two-phase client protocol is used: the client first calls `/router/decide_order` with the unordered, retrieval-order chunk ids and receives back `{worker, ordered_chunk_ids}`; it then issues the real completion request with an `x-router-force-worker` header, and the router rewrites the prompt server-side into the returned order before forwarding. This keeps the reordering decision and the worker decision atomic without a second network hop from the router’s perspective.

1) *Token-weighted cache gain*: An early implementation measured cache gain as the *count* of chunks matched against a worker’s cached prefix, not their token length. Because this project’s corpus preserves natural, variable-length paragraphs (chunks range from 9 to 754 tokens after block-alignment padding, mean 182, $\sigma=108$; Section IV-A), a worker holding two large chunks (say, 1,100 tokens combined) can lose to a worker holding three small ones (300 tokens combined) under count-based accounting, despite the first worker offering nearly four times the real prefill saving. We replaced chunk-count accounting with token accounting, $\text{gain} = \text{hit_tokens}/\text{total_tokens}$, computed by reusing the router’s own tokenizer (never a second, redundant load). A synthetic regression test constructs exactly this few-large-vs-many-small scenario and asserts that the token-weighted and count-weighted computations select different workers, proving the fix has real effect rather than being a no-op relabelling.

2) *Quality protection (`protect_top_k`)*: Pure greedy reordering can push a highly relevant but uncached chunk to the end of the prompt while pulling a barely relevant but cached chunk to the front – a risk CacheWeaver’s own evaluation does not fully rule out (its Table 7 states only that reordering *did not measurably hurt* answer quality in their setting, not that it never can). `protect_top_k` pins the first K retrieval-order chunks in place, unconditionally, and reorders only what follows; $K=0$ reproduces the original behaviour exactly. A regression test constructs a case where the single most relevant retrieved chunk is not cached: with $K=0$ it is demonstrably pushed out of first position by a cached-but-less-relevant chunk, and with $K=1$ it stays first.

D. The cacheweaver_dualmap Baseline and a Tie-Break Defect

DualMap’s official implementation (github.com/ASISys/DualMap, MIT-licensed, which also incorporates Preble) is publicly available, but was not used directly: it is written against an 8+-GPU, 512 GB-DRAM reference deployment and consumes Mooncake-format traces (fixed prefixes, no real retrieval step), neither of which matches this project’s two-machine RAG deployment with genuine per-query retrieval. The routing logic itself – dual-hash candidate selection (Eq. 5), migration benefit (Eq. 6), and SLO-aware routing – was therefore reimplemented independently against this codebase, honestly labelled `cacheweaver_dualmap` throughout this paper as *our DualMap-equivalent implementation*, not as a run of the official release. Our independent reimplementations of

DualMap fixes a bug present in an earlier internal version, where the shared knowledge tree used for CacheWeaver’s reordering decision was also used for DualMap’s per-replica cache-hit estimate; since both candidate replicas were then scored against the *same* tree, the two recompute-cost estimates were always equal, and the router’s cache-affinity term degenerated into always preferring the first candidate until it became overloaded. Giving each replica its own tree fixes this, but exposed a second, subtler defect described in Section VI-F: the candidate comparison used a non-strict inequality ($\text{recompute_r1} \leq \text{recompute_r2}$), so *any* tie – common, since both trees are frequently equally cold or equally warm – was silently resolved in favour of the hash-assigned r_1 candidate rather than broken at random, exactly the class of bias the rest of this codebase’s tie-breaking logic (its `argmax` helper and `per_worker_tree`’s own comparison) was written specifically to avoid.

E. Overlap-Adaptive Weighting

A single signal underlies two independent adaptation mechanisms: the Jaccard overlap $J(A, B) = |A \cap B|/|A \cup B|$ between the retrieved chunk set of the current request and that of the previous one. This is the same definition used by the project’s offline overlap-characterisation tool (Section IV-C), imported rather than reimplemented, so live and offline measurements of “how much locality is available right now” are guaranteed comparable.

Threshold mode. A one-line switch: $\alpha=0.2$, $\beta=0.8$ when J falls below a configured threshold (locality is a weak signal, weight load instead), and $\alpha=0.8$, $\beta=0.2$ otherwise, with $\alpha + \beta=1$ by construction.

EWMA/CUSUM mode. An exponentially weighted moving average tracks a smoothed overlap estimate $D_t = \lambda J_t + (1-\lambda)D_{t-1}$; a CUSUM detector accumulates deviation below a reference level D_{ref} and, once the accumulated statistic exceeds a threshold h , declares a drift event and *recalibrates* D_{ref} to the current EWMA estimate. Without this recalibration step, a detector tuned to a pre-drift regime keeps firing indefinitely in the new regime – a failure mode caught during self-testing of an earlier version, where every request from $t=50$ onward triggered an alarm because D_{ref} was never updated. The recalibrated estimate feeds $\beta_{\text{live}} = \beta_{\text{base}} \cdot \text{clip}\left(1 + k \cdot \frac{D_{\text{target}} - D_t}{D_{\text{target}}}, [0.3, 3.0]\right)$, while α is held fixed – mirroring how the same mechanism is used by `adaptive_cache_aware` to drive β and the guard band δ_0 , but as a fully independent estimator instance with its own state.

Both modes are deliberately fed the *same* consecutive-pair signal (rather than, say, letting the EWMA/CUSUM mode use `adaptive_cache_aware`’s session-adjacent variant) so that a comparison between them isolates the adaptation mechanism as the only variable. Both default to disabled, reproducing the fixed- α, β baseline exactly.

F. Semantic Candidate Pre-Filtering

For deployments with more than a handful of replicas, `per_worker_tree`’s $O(k^2)$ per-candidate reorder becomes

$n \cdot O(k^2)$ across all healthy workers – cheap at $n=2$, not necessarily cheap beyond it. `SemanticWorkerRouter` addresses this independently of locality-based weighting: each worker’s recent query traffic is summarised as an EWMA centroid in embedding space (multilingual-e5-base, the same model used for corpus retrieval, loaded once and shared rather than duplicated); an incoming query is embedded and ranked by cosine similarity against each worker’s centroid, and only the top- k ranked workers are passed to `per_worker_tree`’s comparison. A worker with no observed traffic yet is assigned the *average* similarity of workers that do have a centroid – neither a fixed low score (which would permanently exclude new or recently restarted workers) nor a fixed high score (which would make every cold worker an automatic favourite) – and the candidate set falls back to every healthy worker whenever the semantic favourites happen to be unhealthy, so the mechanism cannot starve a request. Loading the embedding model is forced synchronously at router start-up rather than being paid by whichever live request happens to arrive first.

A documented limitation: this pre-filter is designed to run *before* retrieval, but the live two-phase protocol (Section III-C) currently carries only chunk ids to `/router/decide_order`, not the original query text, so on the live path the filter degrades gracefully to "no filtering" (every healthy worker is a candidate) rather than being exercised. It is validated in isolation (Section VI-H) and end-to-end against live traffic when driven directly with query text; wiring query text through the live HTTP protocol is future work (Section VIII).

IV. IMPLEMENTATION DETAILS

A. Corpus and Trace Construction

The RAG corpus is built from TQuAD, a SQuAD-format Turkish question-answering dataset (681 articles, 8,308 questions). Each question already points at exactly one source paragraph, so using the paragraph as the retrieval chunk gives retrieval ground truth "for free". Paragraphs are split at sentence boundaries only when they exceed a maximum chunk length, and every chunk is padded (in token space, not by appending raw text – tokenizers merge whitespace unpredictably) to a multiple of vLLM’s KV-cache block size, since a chunk boundary that does not align with a block boundary produces a different block hash even when the underlying content is shared, silently breaking the prefix chain the whole system depends on. The resulting corpus contains 2,619 chunks; chunk length after padding ranges from 9 to 754 tokens (mean 182.2, $\sigma=108.5$), confirming that the corpus is *not* fixed-length and motivating the token-weighting fix of Section III-C.1.

Synthetic request traces are generated once per experiment and replayed identically across every strategy under comparison, so that strategies are never compared against different workloads. Two independent knobs control cache locality: popularity skew across titles (a Zipf distribution with configurable exponent s) decides *which* chunks are asked about repeatedly, and session structure (several questions drawn from one title, arriving with realistic think-time gaps) decides *when*

repeats occur relative to each other – a popular chunk asked about ten times over an hour is not locality if the cache empties between hits. An optional popularity-drift knob rotates the ranking over the course of a trace, exercising the scenario an adaptive policy is meant to handle and a static one is meant to degrade gracefully under.

B. Calibration Discipline

TABLE II
MEASURED (NOT GUESSED) SYSTEM CONSTANTS

Constant	Value	Measured via
LOAD_REF	16 concurrent requests	Kleinrock power (throughput/latency) peak, single-worker concurrency sweep, Qwen2.5-7B, ~1700-token prompts
AVG_PROMPT_TOKENS_ESTIMATE	2,392 tokens	Mean prompt token count, $n=800$ live RAG requests, top- $k=10$
DualMap SLO / rebalance thresholds	38,272 / 57,408 tokens	$\text{LOAD_REF} \times \text{mean prompt tokens}; 1.5 \times \text{ratio preserved from DualMap's own SLO/rebalance relationship}$
D_TARGET (drift detector reference)	0.529	Session-adjacent Jaccard overlap mean, $n=800$ live queries
TRACKER_CAPACITY	5,840 blocks	Smaller of the two deployment machines’ real KV-cache block capacity

Table II lists every constant in the system that would otherwise have been a borrowed default (DualMap’s own SLO thresholds are tuned to its authors’ hardware, not ours) or an outright guess (an early version of the prompt-token estimate used 1,700, chosen by inspection rather than measurement). Section VI-B reports the consequence of *not* doing this for TRACKER_CAPACITY: a core ablation’s conclusion inverted once the constant was corrected.

C. Overlap Characterisation Tool

A standalone tool computes the Jaccard overlap between consecutive requests’ retrieved chunk sets directly from a corpus and trace, without touching the router or a live engine (retrieval alone is enough). It reports two distinct notions of "consecutive" separately, because they answer different questions: *session-adjacent* overlap (turn k to turn $k+1$ within one session) reflects what a prefix cache actually sees if the router keeps a session pinned to one worker, while *global-adjacent* overlap (arrival-order adjacency, ignoring session boundaries) reflects the raw opportunity the router faces before any session-affinity policy is applied. This distinction turns out to matter a great deal (Section VI-A).

V. EXPERIMENTAL SETUP

A. Deployment and Protocol

All live experiments use a two-worker vLLM deployment (Qwen2.5-7B-Instruct) connected over a private network,

driven by a replay harness that fires each trace request at its scheduled offset regardless of whether earlier requests have completed – waiting for completions would collapse queue depth to one and prevent the load term of the scoring function from ever activating. Every run validates the router’s own belief about cache state against the engine’s ground truth, read from `usage.prompt_tokens_details.cached_tokens` in the streamed response, rather than assuming the two agree.

B. Evaluation Metrics

The overlap×load sweep matrix described as future work (Section VIII, item 1) is designed to score every cell on five metrics:

- **TTFT (Time to First Token).** The direct indicator of prefill cost – time from request arrival to the first generated token. Reported at p50 (typical behaviour) and p95 (tail behaviour) separately, since a policy can improve one while worsening the other.
- **Throughput / goodput.** The peak sustainable request rate (req/s) under a fixed TTFT SLO (e.g. <5 s), rather than raw throughput alone, so that a policy cannot trade tail latency for a throughput number that would not actually be usable.
- **Cache reuse (token-weighted).** The fraction of prompt tokens served from cache rather than recomputed. Measured in *tokens*, not chunk count – Section III-C.1 shows why chunk-count accounting can pick the wrong worker when chunk lengths vary, which they do substantially in this corpus (Section IV-A).
- **Load balance (CV).** The coefficient of variation of request counts across workers; lower means a more even split.
- **Answer quality (Exact Match / token-F1).** Reordering changes prompt order but not content, and CacheWeaver’s own evaluation (Table 7) found no measurable quality loss from reordering in their setting – but, as noted in Section III-C.2, that is not a universal guarantee. This metric compares retrieval-order and reordered outputs on the identical question set to check whether that guarantee holds here; until it is measured, “reordering preserves quality” should be read as an inherited assumption, not a result of this paper (Section VII, Section VIII).

VI. RESULTS

A. Overlap Characterisation

On a 3,000-request trace (830 sessions, baseline popularity skew $s=1.0$), session-adjacent overlap averaged 0.476 (median 0.500, 90th percentile 1.000; 76.2% of consecutive same-session pairs shared at least one chunk), while global-adjacent overlap averaged only 0.055 (median 0.000; only 7.2% of arrival-order-adjacent pairs shared anything). The gap is the central argument for session-affinity routing: without it, prefix-cache locality is nearly unusable even though it is abundant within sessions.

Sweeping popularity skew from $s=0$ (uniform) to $s=2.0$ (heavily skewed) moves global-adjacent overlap 13-fold, from

0.023 to 0.308 – the more concentrated real-world query popularity is (e.g. FAQ-style repeated questions), the more of a chance a cache-blind router has of stumbling onto a hit even without session affinity. Popularity drift has the opposite, and faster-saturating, effect: turning drift on at all (rate 0.1) drops session-adjacent overlap from 0.476 to 0.383 and global-adjacent overlap from 0.055 to 0.030, with almost no further degradation from 0.1 to 0.5 – direct evidence for why a *static* locality-based policy should be expected to degrade under drift while an adaptive one should not.

B. Calibration Reverses an Ablation

TABLE III
CACHE-AWARE ($\beta=1$) VS. LOAD-BLIND ($\beta=0$), BEFORE AND AFTER TRACKER_CAPACITY RECALIBRATION

Load	$\beta=1$ TTFT p50	$\beta=0$ TTFT p50	Winner (calibrated)
30×	0.256 s	0.488 s	$\beta=1$ (+47%)
35×	2.404 s	2.107 s	$\beta=0$ (+12%)

Before TRACKER_CAPACITY was corrected, the router’s belief about how much of each replica’s cache it could rely on was computed against an assumption roughly nine times larger than either machine’s real capacity. Under that assumption, $\beta=0$ appeared to win at 30× load; after recalibrating capacity to the smaller machine’s real value (5,840 blocks) and re-running under an identical protocol (cold restart before every arm), the winner at 30× flips to $\beta=1$ (Table III). The direction change is not noise: it survives across the two load levels tested and across a second, independently generated trace with different locality characteristics (Section VI-D). The finding this points to is that an uncalibrated capacity constant is not merely a source of quantitative error – it can invert which policy an ablation recommends.

A secondary, still-open observation from the same experiments: worker split is consistently asymmetric in a strategy-dependent, load-independent way ($\beta=1$ runs consistently route more traffic to one physical worker, $\beta=0$ runs to the other). *cache_aware*’s own tie-breaking is already randomised, so this is not the same class of defect identified in Section VI-F; it is more likely attributable to a genuine, unmeasured asymmetry between the two physical machines (network latency, GPU clock state) and is left as an open item (Section VII).

C. *cacheweaver_dualmap* vs. *cache_aware*

TABLE IV
THREE-RUN REPEAT COMPARISON, CALIBRATED THRESHOLDS

Strategy	TTFT p50 (3 runs)	Mean	Hit rate
<i>cache_aware</i>	0.272 / 0.346 / 0.318 s	0.312 s	66.0–66.6%
<i>cacheweaver_dualmap</i>	0.314 / 0.309 / 0.363 s	0.329 s	66.5–66.7%

A single earlier run had suggested a 56% TTFT advantage for *cache_aware*; repeating the comparison three times under calibrated thresholds narrows the gap to 5.5%, with

overlapping variance across runs – the original difference was substantially measurement noise from an $n=1$ comparison, not a real effect. A second, independent difference does survive repetition, however: `cacheweaver_dualmap`’s worker split locks to a near-identical ratio across all three runs ($\sim 538/262$ each time), while `cache_aware`’s split varies considerably run to run (282/518, 293/507, 349/451). Section VI-F identifies the mechanism behind this stability.

D. Sensitivity Analysis

TABLE V
 $\beta=1$ vs. $\beta=0$ ACROSS TRACE LOCALITY AND LOAD

Trace	Load	$\beta=1$	$\beta=0$	Winner
High locality	30×	0.256 s	0.488 s	$\beta=1$ (+47%)
High locality	35×	2.404 s	2.107 s	$\beta=0$ (+12%)
Low locality	30×	0.322 s	0.240 s	$\beta=0$ (+25%)
Low locality	35×	2.018 s	4.370 s	$\beta=1$ (+54%)

Repeating the crossover comparison on a second trace with markedly lower repeat rate (68.8% vs. 91.9% for the original) produces four data points with four different winners and magnitudes (Table V). The clearest pattern is at the highest load tested (35×): the gap between policies grows substantially on both traces, but in *opposite* directions (a 12-point win for one policy on one trace, a 54-point win for the other policy on the other). A claim of the form "policy X wins at high load" is therefore underspecified without also stating which workload locality regime it was measured under – the benefit of the cache-affinity term is a joint function of load *and* locality, not load alone.

E. Dispatch-Time vs. Completion-Time Bookkeeping

TABLE VI
ROUTER-BELIEVED VS. ENGINE-TRUTH CACHE-HIT TOKENS

Recording point	Correlation	Bias	Underestimate rate
Dispatch	0.932	−13.6	2.4%
Completion	0.749	−75.8	14.0%

Recording a block as cached at the moment a request is *dispatched* is, in principle, optimistic – prefill has not actually happened yet, so the belief could be wrong if the request later fails. Recording at *completion* is, in principle, conservative and therefore expected to be more accurate. Under identical trace, load, and calibration conditions, the opposite is observed (Table VI): completion-time recording is both less correlated with engine ground truth and substantially more biased toward underestimating real cache hits. At high concurrency, completion-time bookkeeping misses the contribution of sibling requests that were dispatched to the same prefix but have not yet finished by the time a new request needs to know what is cached – a systematic underestimate that dispatch-time recording, by construction, cannot produce. Much of what an earlier, uncalibrated comparison had attributed to "dispatch-time optimism" is better explained as uncalibrated

capacity (Section VI-B); once capacity is corrected, dispatch-time recording is the more accurate choice in this regime.

F. A Tie-Break Defect and Its Fix

The candidate comparison inside `cacheweaver_dualmap`’s SLO-aware selection step used a non-strict inequality, so that whenever two candidates’ estimated recompute cost tied – common, since both per-replica trees are frequently equally cold or equally warm – the hash-assigned first candidate won unconditionally. We replaced both non-strict comparisons in the selection path with an explicit, uniformly random tie-break, matching the convention already used elsewhere in the codebase. A statistical fairness test constructs 400 requests engineered to tie (fresh, never-before-seen chunk sets against two empty per-replica trees) and records the resulting worker-selection distribution: the corrected code splits 197/203 (49.2%/50.8%); the unmodified code is deterministic and would have produced 400/0 on the same input. This is the most likely explanation for the near-constant worker-split ratio observed across repeated `cacheweaver_dualmap` runs in Section VI-C; confirming that the fix changes the *live*, GPU-measured split (rather than only the synthetic fairness test) is listed as immediate future work (Section VIII).

G. Token-Weighted Cache Gain

On a synthetic scenario constructed so that one worker’s cache holds two large chunks (combined $\approx 1,300$ tokens) and a second worker’s cache holds three small chunks (combined ≈ 9 tokens), count-based cache-gain accounting selects the three-small-chunk worker (3 matched chunks beats 2), while token-weighted accounting selects the two-large-chunk worker ($\text{hit_tokens}/\text{total_tokens} \approx 0.99$ vs. ≈ 0.01) – the two accounting schemes disagree on this input by construction, and only the token-weighted result reflects the real prefill saving available. This effect is measured, not merely theoretical: chunk lengths in the deployed corpus vary by up to two orders of magnitude (Section IV-A).

H. Live Validation: Semantic Pre-Filter

Run against live two-worker traffic (300 requests, 0 failures), the semantic pre-filter’s only measured cost was a one-time ~ 5 – 6 second embedding-model load that, before a start-up warm-up call was added, landed on whichever live request happened to arrive first after a router restart. Forcing this load synchronously at start-up (verified by measuring the warm-up call’s own duration directly against the duration of the first `predict_candidates` call immediately following it) removes the cost from the request path entirely.

I. Live Validation: *per_worker_tree*

The two-phase decide-order protocol was validated against live two-worker traffic for the first time in this round of experiments (an earlier validation used only a single worker, where no real candidate comparison is possible). Across 300 requests: 0 failures, 100% of completions correctly forced to the worker chosen by `/router/decide_order`

(reason=forced_by_decide_order on every request), and real, non-degenerate traffic splitting (130/170) confirming the joint worker-and-order decision is actually exercising both replicas rather than collapsing to one.

VII. DISCUSSION AND LIMITATIONS

The results in Section VI support a general claim about this class of system: *the largest source of measurement error in a cache-aware routing ablation is often not the routing algorithm at all, but an uncalibrated constant the algorithm depends on*. TRACKER_CAPACITY reversed a head-to-head comparison’s conclusion; the dispatch-vs-completion timing result largely dissolved once capacity was corrected; and an $n=1$ comparison between two genuinely competitive policies overstated a 56% gap that repetition narrowed to 5.5%. None of these findings required a new algorithm – they required measuring quantities the system was already implicitly assuming.

Two open items remain unresolved by this work. First, the cache_aware crossover experiments’ worker-split asymmetry (Section VI-B) has no code-level explanation as clean as the tie-break defect that explains cacheweaver_dualmap’s; ruling in or out a physical asymmetry between the two deployment machines requires infrastructure-level measurement (per-machine latency and GPU clock state) outside this paper’s scope. Second, the semantic pre-filter’s live integration is incomplete by design: the two-phase protocol does not yet carry query text, so the filter is validated end-to-end only when driven with query text directly, not through the live HTTP path. Both are listed as future work below rather than claimed as solved.

Scope was also deliberately bounded at several points during development – an EWMA/CUSUM adaptive mechanism was first built and validated as a self-contained, disconnected module before a later, explicit decision to wire it into two independent call sites (adaptive_cache_aware and per_worker_tree’s ewma_cusum mode) as one of two ablation arms rather than as a silent default. We note this because a system with this many independently toggleable mechanisms is easy to over-claim about; every mechanism in this paper defaults to its baseline behaviour and was verified, via unit test, to reproduce that baseline byte-for-byte when disabled.

A. Additional Threats to Validity

Beyond the two open items above, five further limitations are stated plainly rather than left implicit:

- 1) **Two-worker deployment and DualMap’s degeneracy.** All live experiments run on two replicas (a hardware constraint). This is not an incidental limitation for the cacheweaver_dualmap baseline specifically: Section III-C shows its dual-hash candidate selection is provably a no-op at $n=2$, meaning this evaluation cannot exercise DualMap’s own headline advantage (choosing 2 good candidates out of $n>2$) at all – only the SLO-aware/rebalancing layer on top of an already-fixed candidate pair is actually being tested. Scaling

this comparison to $n>2$ replicas is future work, not something this paper’s results speak to.

- 2) **Synthetic traffic.** The overlap characterisation (Section VI-A) and every live experiment use a synthetic trace (popularity-skewed, session-structured) generated over the real TQuAD corpus, not captured production traffic. Absolute values (overlap magnitudes, TTFT numbers) should be expected to shift under real traffic; the qualitative trends this paper argues from (session-adjacent vs. global-adjacent gap, skew and drift effects) are the parts expected to transfer.
- 3) **Tokenizer mode.** prefix_tracker.py supports both an exact HuggingFace tokenizer and a cheap character-bucket approximation (ROUTER_TOKENIZER=approx, the default, kept for dependency-free smoke tests). Every number reported in Section VI was collected with the exact tokenizer; reusing this system for further measurement should confirm that mode explicitly rather than inherit the approximate default silently.
- 4) **Answer-quality validation is not yet performed.** As stated in Section V-B’s metric definitions, the claim that reordering preserves answer quality is inherited from CacheWeaver’s own evaluation, not independently confirmed on this corpus; Section VIII lists this as the most consequential remaining validation step before any reordering mechanism here should be trusted beyond a research setting.
- 5) **Extended components are prototype-calibrated, not traffic-calibrated.** The semantic pre-filter and the EWMA/CUSUM mechanism are both verified by self-test (Sections III-E, III-F) and, for the semantic filter, by live traffic (Section VI-H) – but their tunable parameters (SEMANTIC_TOP_K, the EWMA rate λ , and the CUSUM margin/threshold k, h) were set to reasonable defaults, not measured against real production traffic the way LOAD_REF or AVG_PROMPT_TOKENS_ESTIMATE were (Table II). They are presented as extended evaluation components on top of the paper’s main contribution, not as part of its central, calibrated claims.

VIII. CONCLUSION AND FUTURE WORK

We presented a cache-aware router for RAG-based LLM serving with seven interchangeable strategies, a worker-independent reordering design that avoids a proven degeneracy in dual-hash candidate selection at small replica counts, two independently ablatable overlap-adaptive weighting mechanisms sharing a common input signal, a semantic pre-retrieval candidate filter, and a calibration discipline applied to every constant the system depends on. The calibration discipline itself produced the paper’s most consequential finding: an uncalibrated constant inverted a core ablation’s conclusion, and correcting it also resolved a separate, seemingly unrelated result about cache-bookkeeping timing.

Future work falls into four categories. (1) **Completing the sweep matrix:** an overlap $\times$ load $\times$ strategy sweep runner (3 overlap levels $\times$ 3 load levels $\times$ 7 strategies, 63 cells) is

implemented and verified GPU-free via a dry-run mode, but has not yet been executed against live hardware. **(2) Closing the semantic pre-filter’s live integration gap** by extending the two-phase protocol to carry query text. **(3) Resolving the open worker-split asymmetry** in `cache_aware` through infrastructure-level measurement, and confirming that the tie-break fix of Section VI-F changes the *live*, GPU-measured `cacheweaver_dualmap` split and not only the synthetic fairness test. **(4) Answer-quality evaluation:** every reordering mechanism in this paper (`protect_top_k` included) currently optimises for prefill latency under the assumption, inherited from CacheWeaver, that chunk order does not materially affect answer quality; measuring that assumption directly against this project’s own TQuAD-derived gold answers is the natural next step before any of these mechanisms should be deployed beyond a research setting.

REFERENCES

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. <https://arxiv.org/abs/2309.06180>.
- [2] K. Tan, R. Gu, and M. Li, “CacheWeaver: Cache-Aware Evidence Ordering for Efficient Grounded RAG Inference,” *arXiv preprint arXiv:2606.19667*, 2026. <https://arxiv.org/abs/2606.19667>.
- [3] Y. Yuan, P. Zuo, B. Wang, Z. Chen, Z. Tan, and Z. Yu, “DualMap: Enabling Both Cache Affinity and Load Balancing for Distributed LLM Serving,” in *Proc. International Conference on Learning Representations (ICLR)*, 2026. <https://arxiv.org/abs/2602.06502>. Code: <https://github.com/ASISys/DualMap>.
- [4] A. Dhasade, R. Guerraoui, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, “Efficient Federated Search for Retrieval-Augmented Generation Using Lightweight Routing,” in *Proc. 5th Workshop on Machine Learning and Systems (EuroMLSys)*, 2025. <https://arxiv.org/abs/2502.19280>.
- [5] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang, “Preble: Efficient Distributed Prompt Scheduling for LLM Serving,” in *Proc. International Conference on Learning Representations (ICLR)*, 2025. <https://arxiv.org/abs/2407.00023>.
- [6] R. Qin, Z. Li, W. He, M. Zhang, Y. Yang, W. Zheng, and Z. Zha, “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving,” in *Proc. USENIX Conference on File and Storage Technologies (FAST)*, 2025. <https://arxiv.org/abs/2407.00079>.
- [7] NVIDIA, “Dynamo: A Datacenter Scale Distributed Inference Serving Framework,” 2025. <https://github.com/ai-dynamo/dynamo>.
- [8] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, and X. Jin, “RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation,” *ACM Transactions on Computer Systems*, vol. 44, no. 1, 2026. <https://arxiv.org/abs/2404.12457>.
- [9] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion,” in *Proc. European Conference on Computer Systems (EuroSys)*, 2025. Best Paper Award. <https://arxiv.org/abs/2405.16444>.
- [10] S. Lu, H. Wang, Y. Rong, Z. Chen, and Y. Tang, “TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 6588–6601. <https://arxiv.org/abs/2410.07590>. Code: <https://github.com/MooreThreads/TurboRAG>.
- [11] S. Agarwal, S. Sundaresan, S. Mitra, D. Mahapatra, A. Gupta, R. Sharma, N. J. Kapu, T. Yu, and S. Saini, “Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation,” *Proc. ACM on Management of Data (SIGMOD)*, vol. 3, no. 3, 2025. <https://arxiv.org/abs/2502.15734>.
- [12] A. Ricci Toniolo, A. Dinesh, and R. Thorstenson, “GORGO: Maximizing KV-Cache Reuse While Minimizing Network Latency in Cross-Region LLM Load Balancing,” *arXiv preprint arXiv:2602.11688*, 2026. <https://arxiv.org/abs/2602.11688>.
- [13] F. Bang, “GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings,” in *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023. <https://aclanthology.org/2023.nlposs-1.24/>.
- [14] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu, “SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models,” *arXiv preprint arXiv:2406.00025*, 2024. <https://arxiv.org/abs/2406.00025>.
- [15] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, “MeanCache: User-Centric Semantic Caching for LLM Web Services,” *arXiv preprint arXiv:2403.02694*, 2024. <https://arxiv.org/abs/2403.02694>.